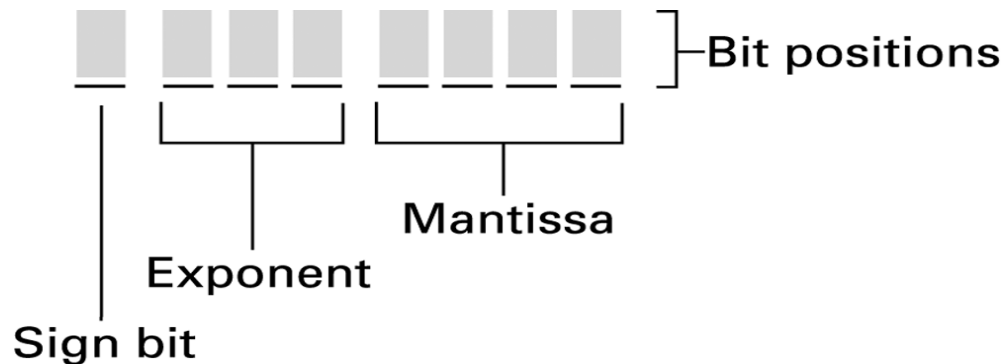


Representing Floating Point Numbers in Binary

We are going to represent floating point numbers using 8-bits, with the following format:



As you will see, 8-bits only allows us to represent numbers with a small amount of relative accuracy and magnitude. In modern computer systems, 32 or 64 bits are used to store floating point numbers. We use 8-bits so that we can easily illustrate the concept, and easily convert numbers to and from floating point format.

To convert to binary floating point format, express the binary number in normalized form. To convert a binary number to normalized form, move the radix point to the left of any digit with the value of one (1) and record the exponent based on the number of digits you moved the radix point. Left is positive, right is negative (just like with base-10 numbers).

General Normalized Form: $0.1??? \times 2^{\text{exponent}}$

Mantissa: The mantissa is the 4 bits to the right of the radix point.

Exponent: The exponent is expressed in excess-4 notation:

Binary	Subtract	Excess-4 Value
000	0 - 4	-4
001	1 - 4	-3
010	2 - 4	-2
011	3 - 4	-1
100	4 - 4	0
101	5 - 4	1
110	6 - 4	2
111	7 - 4	3

For example, 10.11 in normalized form is: 0.1011×2^2 ; so the mantissa is 1011 and the exponent is 110
And 0.0001011 in normalized form is: 0.1011×2^{-3} ; so the mantissa is 1011 and the exponent is 001

Sign Bit: The sign bit is 0 if the number is positive, or 1 if the number is negative.

Overflow results when the exponent cannot be represented by excess-4 notation.

Round-off, truncation, or loss of precision happens when the mantissa does not fit in 4 bits.

Example 1: Convert $2 \frac{3}{8}$ to binary floating point format.



The linked image cannot be displayed. The file may have been moved, renamed, or deleted. Verify that the link points to the correct file and location.

$$= (10.011)_2$$

Normalized form: 0.10011×2^2

Mantissa: 1001 (loss of precision occurs)

Exponent: 110

Sign: 0

In binary floating point format: $2 \frac{3}{8}$ is approximately 01101001

Convert back to base-10:

Sign: positive

Exponent: 2

Mantissa: 1001

Normalized form: $0.1001 \times 2^2 = 10.01 =$



(Notice the effect of loss of precision)

Example 2: Convert -0.0123 to binary floating point format.

Using the multiply by 2 method to convert decimal fraction to binary fraction:

$$0.0123 \times 2 = 0.0246$$

$$0.0246 \times 2 = 0.0492$$

$$0.0492 \times 2 = 0.0984$$

$$0.0984 \times 2 = 0.1968$$

$$0.1968 \times 2 = 0.3936$$

$$0.3936 \times 2 = 0.7872$$

$$0.7872 \times 2 = 1.5744$$

$$0.5744 \times 2 = 1.1488$$

$$0.1488 \times 2 = 0.2976$$

$$0.2976 \times 2 = 0.5952$$

$$0.5952 \times 2 = 1.1904$$

$$0.1904 \times 2 = 0.3808$$

So $-0.0123 = -0.000000110010\dots$

Normalized form: $0.110010\dots \times 2^{-6}$

Mantissa: 1100 (loss of precision occurs)

Exponent: Overflow occurs. Cannot represent -6 in excess notation. Due to the "number circle", -6 will be represented as 110 (in excess notation, $-6 = -4 - 2$; and 2 places below -4 is positive 2)

Sign: 1

In binary floating point format: $-0.0123 = 11101100$

Convert back to base-10:

Sign: negative

Exponent: 2

Mantissa: 1100

Normalized form: $-0.1100 \times 2^2 = -11.00 = -3.0$ (Notice the effect of overflow)